Huellita

Documentación del sistema e informe de avance

Proyecto final UTN
Fecha de corte 6 de noviembre de 2026

Equipo del proyecto
Laureano Beltrami · Francisco Bareuther
Leon Comolli · Luciano Brea

Resumen del estado actual

Huellita reúne a refugios y personas interesadas en adoptar en una plataforma web común. El sistema integra un catálogo público, gestión de animales, solicitudes, adopciones, historia clínica y comunicación entre el refugio y la familia adoptante.

El código actual contiene el recorrido de adopción y módulos de chat y seguimiento posterior. Las pruebas automatizadas y la compilación fueron satisfactorias. La instalación local todavía requiere aplicar seis migraciones para alinear su base de datos con esas funciones; por eso la disponibilidad del código no equivale a una validación integral de esa instalación.

Comprobación	Resultado al corte
Pruebas del servidor	**138 aprobadas** y 3378 aserciones en SQLite en memoria
Pruebas de presentación	**45 aprobadas** en la suite frontend
Compilación de producción	**Correcta** con TypeScript y Vite
Base local de MariaDB	**8 migraciones aplicadas y 6 pendientes**

Propósito y organización

Este documento sirve como referencia para el equipo y para la presentación del avance académico. Describe el alcance funcional, las reglas de negocio, la arquitectura, las mejoras recientes, la operación local y las tareas necesarias para preparar una entrega verificable.

Contenido 2 Alcance y roles · 3 Procesos principales · 4 Arquitectura y datos · 5 Avance de la interfaz · 6 Validación · 7 Operación local · 8 Próximos hitos · 9 Referencias

Referencia de código: rama local **main**, commit base **e9d18db**, con cambios de interfaz y dependencias aún sin confirmar en Git.

Alcance funcional y roles

La plataforma trabaja con varios refugios. Cada animal pertenece a una institución y el personal administra los registros de su propio refugio. Las personas adoptantes usan una cuenta global para consultar animales y realizar solicitudes en distintas instituciones.

Perfil	Responsabilidades y alcance
Visitante	Consulta portada, animales, refugios y perfiles públicos. Puede enviar una consulta de contacto. No es un rol persistido.
Adoptante	Gestiona sus solicitudes, conversa con el refugio y presenta los controles de seguimiento de sus adopciones.
Administrador	Gestiona animales, fotografías, solicitudes, adopciones y salud del refugio al que pertenece. Revisa los controles posteriores.
Superadministrador	Administra refugios y personal, consulta indicadores y registros de la red y recibe consultas de contacto. No hereda permisos para aprobar adopciones o editar salud.

Módulos presentes en el código

Módulo	Funciones principales
Sitio público	Portada, explicación del proceso, equipo, contacto, directorio de refugios y créditos de fotografías.
Catálogo y animales	Filtros por nombre, especie, raza, tamaño, edad y disponibilidad; ciudad y refugio en el catálogo general. Perfiles y galerías.
Cuentas y refugios	Registro, sesión y recuperación de contraseña. Alta manual de refugios y formulario público de registro de institución con administrador.
Adopciones	Presentación y evaluación de solicitudes, reserva, adopción definitiva e historial de decisiones.
Salud	Consultas, vacunas, tratamientos, controles, agenda de dosis y auditoría de correcciones.
Comunicación y seguimiento	Chat por solicitud, plazos de respuesta, controles con fotografías y certificados de vacunación.

El registro público de refugios crea una institución activa y una cuenta administradora. Este comportamiento figura en el controlador actual y debe reflejarse en la documentación y en la política de incorporación de instituciones.

El alcance actual no incluye pagos, donaciones, traslados de animales entre refugios ni un flujo de cancelación de adopciones definitivas y readopción.

Procesos principales del sistema

Solicitud y adopción



Recorrido habitual. Nueva también puede pasar directamente a Aprobada.

El adoptante elige un animal disponible y presenta el formulario. El servidor verifica el rol, la actividad del refugio y la ausencia de otra solicitud activa de la misma persona para ese animal. El refugio evalúa y registra cada decisión con sus observaciones.

Una solicitud aprobada reserva al animal y cambia su disponibilidad a **En proceso de adopción**. La adopción se concreta mediante una acción posterior: se crea el registro definitivo, el animal pasa a **Adoptado** y las demás solicitudes activas se cierran con un motivo visible.

- Nueva, En revisión y Aprobada pueden terminar en Rechazada. Rechazada y Adoptado no tienen transiciones posteriores en este flujo.
- Si una solicitud aprobada se rechaza antes de concretarse, el animal vuelve a estar disponible.
- Las transacciones, los bloqueos de filas y las restricciones de unicidad protegen las decisiones concurrentes y evitan dos adopciones definitivas para el mismo animal.

Comunicación entre la familia y el refugio

Cada solicitud dispone de una conversación privada. Cuando el refugio escribe y espera respuesta durante el proceso previo a la adopción, el servicio genera un aviso a las 24 horas y cierra la solicitud por inactividad a las 48 horas. Una respuesta del adoptante detiene ese plazo. Tras la adopción, la conversación continúa sin ese vencimiento.

La tarea **adoptions:expire-inactive** está programada cada quince minutos. Algunas vistas también comprueban los plazos. Para que el cierre ocurra sin visitas a la aplicación, el programador de Laravel debe estar operativo.

Seguimiento posterior y cuidados

Al concretarse una adopción se programan **nueve controles**: cuatro semanales durante el primer mes y cinco mensuales entre el segundo y el sexto mes. Cada control solicita una foto de las categorías Jugando, Durmiendo, Comiendo y Convivencia; se habilita siete días antes de su vencimiento.

La familia envía las fotografías y un comentario opcional. El refugio registra la revisión y sus observaciones. Los certificados de vacunación aceptan imágenes o PDF. Estos archivos se guardan en almacenamiento privado y se entregan mediante rutas autorizadas.

La salud del animal conserva consultas, tratamientos, vacunas, controles y auditorías. Una dosis prevista requiere registrar explícitamente su aplicación o cancelación. Los antecedentes de animales archivados se consultan en modo de solo lectura.

Arquitectura y organización de los datos

La solución es un monolito Laravel con interfaz React y TypeScript. Inertia conecta los controladores con las páginas del navegador; las rutas de la aplicación gestionan sesiones y formularios. Vite compila los recursos y Tailwind define gran parte de los estilos.



El servidor valida los datos, aplica permisos y persiste cada operación.

Tecnología	Versión o uso verificado
Laravel e Inertia para Laravel	12.70.0 y 2.0.28 en composer.lock
React y TypeScript	18.3.1 y 5.9.3 instalados
Vite y Tailwind CSS	7.3.7 y 4.0.17 instalados
GSAP y Lucide React	3.15.1 y 1.61.0 instalados
Persistencia	Conexión mysql hacia MariaDB local; SQLite en memoria para las pruebas de este informe

Entidades y relaciones principales

Grupo	Relaciones y finalidad
Refugios y usuarios	shelters organiza las instituciones. users distingue tres roles y vincula al personal con su refugio.
Animales y fotografías	animals pertenece a un refugio. animal_photos contiene sus imágenes; el archivo lógico conserva antecedentes.
Solicitudes y adopciones	adoption_requests vincula persona y animal. adoptions registra la adopción definitiva; adoption_request_history conserva las transiciones.
Salud y trazabilidad	Consultas, vacunas, tratamientos y controles se vinculan al animal. vaccination_doses registra cumplimiento; health_audits conserva cambios.
Comunicación y seguimiento	Mensajes asociados a solicitudes, controles y fotos asociados a adopciones, certificados y consultas del formulario de contacto.

Permisos y archivos

Los middleware, policies y servicios comprueban el rol y la pertenencia al refugio. Las imágenes públicas de animales e instituciones se separan de los archivos privados del seguimiento. El código distingue al profesional que atendió del usuario que registra o modifica la historia clínica.

Avance reciente de la interfaz y el entorno

El trabajo reciente se concentró en recuperar el entorno local y mejorar la transición de la portada hacia la explicación de la plataforma. El diseño final conserva el fondo claro de la marca y utiliza movimiento asociado al scroll.

Hero con interacción ligera

Las seis cabezas decorativas tienen una flotación suave con ritmos diferentes. Con un puntero compatible, el hover agrega un pequeño aumento de tamaño y un giro. La animación se implementó con CSS y transformaciones; respeta la preferencia de movimiento reducido del dispositivo.

Transición hacia Conocé Huellita

Se instaló GSAP y se incorporó ScrollTrigger. Mientras avanza el scroll, el hero permanece fijo y la sección siguiente sube por delante. El texto, los botones y las cabezas pierden opacidad de forma progresiva y reaparecen cuando el usuario vuelve hacia arriba.

El comportamiento está concentrado en HeroAboutTransition. El pin se aplica solamente al hero para conservar el flujo del contenido y los pasos sticky de la sección informativa. En ventanas de hasta 560 píxeles de alto o con movimiento reducido se mantiene el desplazamiento normal. La animación se limpia cuando el componente se desmonta.

Terminación visual y legibilidad

El fondo de Conocé Huellita volvió al tono claro original. La entrada utiliza un borde difuminado claro y la base del hero termina en el mismo color. Las fotografías conservan el texto inferior izquierdo; se retiró la etiqueta “Fotografía ilustrativa”.

El fondo de los textos sobre la foto se redujo de 74 % a 60 % de opacidad y el desenfoque de 16 a 3 píxeles. Los bordes se extienden gradualmente, el texto es blanco y tiene una sombra leve. El cálculo sobre una imagen blanca da una relación de contraste aproximada de 4,59 a 1 en la zona de cobertura completa; falta completar la revisión visual de todas las fotos y tamaños.

Incidencias resueltas

Incidencia	Acción y resultado
Conexión rechazada en 3307	Se inició la instancia local de MariaDB que utiliza los datos del proyecto. Se verificó respuesta HTTP 200 en inicio y login durante la intervención.
Importación de lucide-react	La dependencia estaba declarada, pero ausente de node_modules. Se instaló y Vite volvió a resolver el componente.

La compilación actual informa 59,65 kB gzip para el bloque Home y 119,50 kB gzip para el bloque compartido app. Son tamaños de salida de Vite, no mediciones de velocidad percibida o de consumo en dispositivos.

Validación y estado de la instalación local

Verificación	Resultado	Alcance
Pruebas PHP	138 aprobadas 3378 aserciones	SQLite en memoria, entorno testing y configuración PHP local heredada por los subprocesos.
Pruebas frontend	45 aprobadas	Contratos de presentación y componentes renderizados por las pruebas.
TypeScript y Vite	Compilación correcta	Generación de recursos de producción; advertencia sobre la resolución de una imagen del contacto.
Esquema MariaDB	8 aplicadas 6 pendientes	Inspección con migrate:status. No se aplicaron migraciones durante la elaboración del informe.

La primera ejecución PHP aprobó 135 pruebas y falló en tres generadores de imágenes por falta de GD en el proceso de pruebas. Al propagar la configuración local mediante PHPRC, la suite completa aprobó las 138 pruebas. Esto identifica un requisito de configuración del entorno, no una corrección de lógica del producto.

Migraciones pendientes en la base local

Fecha y cambio	Impacto funcional
27 de octubre contact_messages	Persistencia de las consultas del formulario de contacto.
28 de octubre phone en contact_messages	Teléfono de la persona que envía una consulta.
30 de octubre Enlaces sociales de shelters	Sitio web, Instagram y Facebook del refugio.
30 de octubre Tipo y medicación de treatments	Ampliación de la información de tratamientos.
30 de octubre Chat y seguimiento	Mensajes, plazos, controles, fotos y certificados posteriores a la adopción.
30 de octubre Marcadores de lectura	Lectura y recuento de mensajes en las conversaciones.

Los módulos pueden superar pruebas sobre una base creada desde cero y fallar en una instalación que todavía no tiene las tablas o columnas nuevas. La actualización del esquema local es, por tanto, el primer pendiente operativo.

Límites de la evidencia

Las pruebas no sustituyen la revisión visual, las mediciones de rendimiento ni la validación del flujo completo contra MariaDB. La comprobación visual automatizada de los últimos cambios quedó pendiente. Tampoco se acredita un despliegue público, una ejecución remota de CI o el envío de correo real.

Ejecución local y mantenimiento

El directorio de trabajo es D:\proyectofinal. El entorno preparado incluye MariaDB en 127.0.0.1:3307, Laravel en el puerto 8000 y Vite para desarrollo. El script local administra esos procesos. El MySQL de XAMPP en 3306 es una instancia distinta.

Arranque de esta instalación

```
cd D:\proyectofinal
.\.local\dev.cmd
# Abrir http://127.0.0.1:8000
# Al terminar: .\.local\stop.cmd
```

Para una nueva computadora se requieren PHP 8.2 o superior con las extensiones de Laravel, PDO MySQL y GD para las pruebas con imágenes; Composer, Node y un servidor MySQL o MariaDB. La conexión debe corresponder al servicio de esa computadora, sin asumir el puerto 3307 de esta instalación.

Actualización de una instalación existente

- Conservar .env, APP_KEY y Storage. Realizar un respaldo de base y archivos y comprobar su recuperación en una copia.
- Instalar dependencias desde los lockfiles con composer install y npm ci. Revisar primero php artisan migrate:status en la conexión correcta.
- Aplicar las migraciones pendientes en una copia de validación, comprobar los recorridos y luego actualizar la instalación acordada mediante php artisan migrate.
- Compilar con npm run build y revisar sesión, imágenes, contacto, adopciones, chat y seguimiento. No usar migrate:fresh o seeders para actualizar datos existentes.

Pruebas reproducibles en PowerShell

Ejecutar las pruebas en una terminal destinada a testing y cerrarla al terminar para no reutilizar sus variables al servir la aplicación. PHPRC permite que los procesos hijos utilicen las mismas extensiones PHP.

```
$env:PHPRC='D:\proyectofinal\.php-local.ini'
$env:APP_ENV='testing'
$env:DB_CONNECTION='sqlite'
$env:DB_DATABASE=':memory:'
$env:DB_URL=''
& C:\xampp\php\php.exe artisan test
npm.cmd run test:frontend
npm.cmd run build
```

Tareas operativas

Configurar el programador de Laravel para las revisiones de inactividad, definir el transporte de correo, respaldar el disco público y el privado y conservar los secretos fuera del repositorio. El primer superadministrador se crea explícitamente con php artisan shelter:create-staff. Las migraciones no generan cuentas privilegiadas.

Pendientes y próximos hitos

El siguiente avance debe consolidar la ejecución local de las funciones implementadas y reunir evidencia de uso real. Las prioridades siguientes se expresan por resultado verificable; no se asignan porcentajes de avance sin una línea base de requisitos aprobada.

Prioridad	Tarea	Criterio de cierre
Alta	Alinear la base local	Aplicar las seis migraciones tras respaldo y prueba en copia. migrate:status sin pendientes y recorridos funcionales satisfactorios.
Alta	Validar el flujo completo	Refugio, animal, solicitud, aprobación, adopción, conversación y primer control probados con dos refugios y usuarios de prueba.
Alta	Actualizar la documentación	Corregir las referencias que excluyen chat y registro público de refugios; documentar los plazos y el seguimiento actuales.
Media	Completar la revisión visual	Comprobar portada, scroll, hover, contraste, anclas y teclado en escritorio, móvil, ventanas cortas y movimiento reducido.
Media	Revisar recursos y desempeño	Resolver la advertencia de la imagen del contacto y medir carga, estabilidad y fluidez con herramientas de rendimiento.
Media	Consolidar la entrega en Git	Agrupar los cambios actuales, revisar dependencias y presentar el cambio mediante una rama y revisión de equipo.
Antes de publicar	Preparar operación	Definir servidor, HTTPS, correo, programador, almacenamiento persistente, respaldos, responsables y recuperación probada.

Recorrido de aceptación sugerido

Un administrador carga un animal y su información sanitaria. Una persona lo encuentra en el catálogo general y en el del refugio, presenta la solicitud y recibe respuesta. El refugio evalúa, aprueba y concreta la adopción. La familia accede al seguimiento, envía las cuatro fotos y el refugio las revisa. Se comprueba que otro refugio y otra familia no puedan consultar esos registros privados.

Conclusión del avance

El proyecto cuenta con una base funcional amplia y con pruebas automatizadas satisfactorias sobre un entorno aislado. Las mejoras recientes de la portada están implementadas y compiladas. El cierre de la siguiente etapa depende de actualizar el esquema local, comprobar la experiencia completa y sincronizar la documentación con el alcance real del código.

Referencias del proyecto

Las referencias siguientes permiten localizar la implementación y reproducir las comprobaciones. Las rutas son relativas a D:\proyectofinal. El estado descrito corresponde a la copia de trabajo y a la fecha de corte indicadas en la primera página.

Funcionalidad y reglas de negocio

routes/web.php
Rutas públicas, registro de refugios, permisos por rol, chat y seguimiento.

app/Services/AdoptionService.php
Transiciones, reserva, adopción definitiva, historial y cierre por inactividad.

app/Services/AdoptionChatService.php
Mensajes, aviso a las 24 horas y cierre a las 48 horas.

app/Services/FollowUpService.php
Calendario de nueve controles, categorías de fotos y almacenamiento privado.

app/Services/HealthService.php
Registros de salud, dosis, validaciones y auditoría.

app/Http/Controllers/ShelterController.php
Alta manual, registro público, personal e imágenes institucionales.

app/Policies y app/Models/Concerns
Autorización por recurso y separación de datos entre refugios.

Interfaz y recursos

resources/js/Components/ui/
FloatingAnimalsHero.tsx, HeroAboutTransition.tsx y About.tsx contienen la portada y la transición. ContactWithDog.tsx y WhoWeAre.tsx contienen contacto y presentación del equipo.

resources/css/app.css y resources/css/about.css
Flotación de las cabezas, degradados, controles y presentación de fotografías.

Configuración y comprobaciones

composer.lock y package-lock.json registran las versiones reproducibles. **phpunit.xml** y **tests/TestCase.php** definen y protegen el entorno de pruebas. **database/migrations/** contiene las catorce migraciones inspeccionadas.

.local/informe-php-tests-verificado.log, **.local/informe-frontend-tests.log** y **.local/informe-build.log** conservan la evidencia de esta revisión en el entorno local. El informe omite contraseñas, claves y datos personales de las cuentas.

Guías complementarias

docs/06-instalacion.md y **docs/14-operacion-y-publicacion.md** describen instalación y mantenimiento. **docs/04-base-de-datos.md** y **docs/13-salud-agenda-entrega.md** amplían datos y salud. **CONTRIBUTING.md** establece el trabajo mediante ramas y revisión.

Las guías históricas que describen chat, seguimiento o registro público como funciones ausentes deben actualizarse. Para esas funcionalidades, este informe toma como referencia las rutas, controladores, servicios y pruebas presentes al corte.